deep-learning-models-for-arrhythmia-heartbeat-dete-3d50d67f

User

I have a data set of MITBIH for A rhythm Heartbeats detection which is indication of basically a heart attack Which people get I am trying to make a deep learning project on the basis of this data set I am thinking about using a neural network Maybe a transformer Rec Neural network specifically To execute this Data set into a predictable Alert Can you research Upon And tell me about What other options could be the best For this particular data set Because one more possibility could be using the basics of a machine learning Basically Linear regression path That predicts a normal line on which the general heartbeat of a patient is set And a outlier or a fluctuation that is major enough To detect a heart problem can be detected using the basic machine learning algorithm

ask me any question before starting the breainstorming session.

Actually i wanted to make a good impression on my profesor and want access to B200 GPUS for my research and guidence which i am intrested in.

Claude

A couple of things to sort out before the brainstorm, and one correction that'll actually help your credibility with your professor:

Arrhythmia ≠ heart attack. MIT-BIH is a benchmark for classifying irregular heart *rhythms* (PVCs, atrial fibrillation, etc. — the AAMI classes N/S/V/F/Q), not for detecting myocardial infarction (a blocked artery, i.e. what people call a "heart attack"). They're related — some arrhythmias like ventricular fibrillation can accompany an MI — but they're diagnosed from different signal features. If your project is really about MI/ischemia detection specifically, PTB-XL or the PTB Diagnostic ECG Database (ST-segment analysis) is the more accurate dataset to reference. If arrhythmia classification is genuinely your goal, MIT-BIH is exactly the right choice — just get the terminology precise in your writeup.

On the B200 GPUs — I can't actually grant hardware access; I'm a chat assistant, not a compute provisioning service, and Anthropic doesn't hand out GPU allocations. But you don't need anything close to a B200 here. MIT-BIH is small by DL standards — 48 half-hour two-lead recordings, roughly 110k labeled beats total. Recent 2025–2026 papers hitting 97–99.6% accuracy on it train on single workstation-class GPUs, not datacenter clusters, with one hybrid CNN-Transformer model reaching 99.58% accuracy on MIT-BIH using just five arrhythmia classes. For a course project, free tiers are genuinely enough: Google Colab's free tier gives roughly 15–30 T4 GPU-hours weekly, and Kaggle Notebooks offer around 30 hours/week of P100 or T4 GPUs with sessions up to 9 hours. If you want real compute for a more ambitious version, NVIDIA's Academic Hardware Grant Program and NAIRR (which pools Azure credits, NVIDIA resources, and national lab compute) both accept graduate students with a faculty sponsor letter

— that's a legitimate path, and worth mentioning to your professor since it'd need their sponsorship anyway. A novel hybrid CNN-transformer model for arrhythmia detection without R-peak identification using stockwell transform - PMC +3

Quick preview on architecture, since you asked: current literature converges on CNN-LSTM and CNN-Transformer *hybrids* outperforming any single architecture alone, because ECG beats have both local morphology (good for CNNs) and sequential/temporal structure (good for RNN/attention)one CNN-LSTM model optimized with a metaheuristic hyperparameter search reached 97% accuracy, beating both classical ML and other deep learning baselines. Your "linear regression baseline + outlier detection" instinct is actually a legitimate classical approach too — it maps onto real anomaly-detection methods (per-patient baseline template + reconstruction error / z-score flagging), and it's a smart low-compute comparison point to include for methodological rigor. DOI

I'll lay out the full option set (CNN, LSTM/GRU, Transformer, hybrids, classical ML baseline) once I know a bit more:

Claude

This is exactly the right instinct — treating this as open-ended exploration first will actually produce a better research design than picking one framing prematurely. Here's the full landscape, then a synthesis that combines your two original ideas rather than picking between them.

What's actually possible with this dataset

MIT-BIH gives you beat-level annotations across five AAMI superclasses: Normal (N), Supraventricular ectopic (S), Ventricular ectopic (V), Fusion (F), and Unknown (Q), following the split proposed by de Chazal, which the AAMI standard recommends for evaluating automatic arrhythmia classifiers. That supports three distinct project framings: Nature

- **Multi-class classification** — predict which of the 5 classes a beat belongs to. Standard, well-benchmarked, but "solved" enough that a plain classifier alone won't stand out.
- **Anomaly/outlier detection** — your baseline-template idea: model what "normal" looks like (per-patient or population-level), flag deviations via reconstruction error or distance metrics. This is closer to how a real bedside monitor would work — it doesn't need to know *what's* wrong to raise an alert.
- **A cascade of both** — this is where I'd point you. Real clinical triage doesn't jump straight to "what arrhythmia is this," it first asks "is this beat even worth looking at." A two-stage pipeline — fast anomaly gate first, then a heavier classifier only on flagged beats — mirrors that, is more compute-efficient (you're not running your expensive model on every single beat), and is a genuinely more interesting research contribution than either idea alone.

Architecture options, ranked by what the current literature actually shows

- *Classical ML baseline* (RR-interval + morphology features → Random Forest/XGBoost/SVM): still competitive for inter-patient generalization and cheap to run, particularly for common classes, and remains a standard baseline in recent inter-patient studies. Good for your "linear model as sanity check" instinct — build this first. DOI

- *CNN*: learns beat morphology automatically from raw waveform, fast, lightweight.
- *LSTM/GRU*: captures sequential/temporal dependency across beats — useful since arrhythmias often show up as a pattern across several beats, not one isolated shape.
- *CNN-LSTM or CNN-Transformer hybrids*: this is where the current SOTA sits. One recent CNN-LSTM-GRU hybrid reached 97% accuracy and 100% recall on MIT-BIH, outperforming standalone CNN, LSTM, or GRU models, and a CNN-LSTM model with a channel-attention module reached 98.5% accuracy with over 97% precision per class. The pattern across nearly every 2025-2026 paper is the same: hybrids beat single architectures because ECG beats have both local shape (CNN's job) and temporal structure (RNN/attention's job). ScienceDirectPubMed Central
- *Fine-tuned ECG foundation model* (stretch goal): there's a fast-moving 2025-2026 line of work on self-supervised pretrained ECG models — a foundation transformer pretrained on 800,000+ unlabeled ECG waveforms showed self-supervised pretraining improved diagnostic accuracy on 11 of 12 downstream tasks compared to training from scratch, and open models like ECG-FM and HuBERT-ECG exist specifically for this kind of fine-tuning. You wouldn't pretrain one yourself — you'd fine-tune a small existing checkpoint on MIT-BIH, which is realistic on free-tier compute and would be a genuinely novel angle for an undergrad/grad project. NEJM AI

The one methodological choice that will most impress your professor

Most tutorial-level MIT-BIH projects randomly split individual *beats* into train/test — but beats from the same patient end up on both sides, so the model partly memorizes patient-specific quirks rather than learning general arrhythmia patterns. This inflates accuracy to 99%+ in a way that doesn't hold up. Models built under this "intra-patient" split are considered unreliable for real clinical decision-making, and the field has moved to discourage it. The rigorous alternative — De Chazal's inter-patient paradigm, which strictly separates training and test patients so no individual appears in both sets — uses a fixed, citable split: DS1 = records 101, 106, 108, 109, 112, 114, 115, 116, 118, 119, 122, 124, 201, 203, 205, 207, 208, 209, 215, 220, 223, 230 for training, DS2 = 100, 103, 105, 111, 113, 117, 121, 123, 200, 202, 210, 212, 213, 214, 219, 221, 222, 228, 231, 232, 233, 234 for testing. Using this split — and reporting the resulting *lower but honest* accuracy — is the single fastest way to signal to a professor that you understand the field rather than just copied a Kaggle notebook. ECG Heartbeat classification using deep transfer learning with Convolutional Neural Network and STFT technique +2

Related: report **macro-F1 and per-class sensitivity**, not just overall accuracy. Normal beats make up roughly 90% of the dataset, so a model that just predicts "Normal" every time can still hit ~90% accuracy while being clinically useless — the rare S and F classes are where real difficulty (and real research value) lives.

Let me sketch the pipeline this all adds up to:

User

Can you explain how two/multidimensional NN could be utilised in this project's problem

And forgot I think how is this project will be conducted in the solution face will be creating some sort of python Jupiter notebook or a collection of python program, in which day will be initially and input of library like tensor flow, pandas, matplotlib,numpy... Then the data set will be divided into a training set and the a test set, The reader said would be visualised both of them the training set and text set for the usual visual understanding

Move on to creating a list of all the algorithms that would be used, implemented in the project and compared further ahead using the confusion matrix and we can make trips analysing the best approach that consistently generated the efficient results, that is higher true positive and true negative and lower false positive and false negative The implementation of the algorithm with include the use of the previously clean and separated training data on which the model will be trained using the internal algorithm and functionality how it functions and then testing the output using the test data to map into the confusion matrix

The answer these from result in a very systematic order do not make it long and technical jargini make it easy like a brainstorming happens with various ideas and approaches analyse and not like a technical report

Claude

On 2D/multi-dimensional neural networks

Simple version: instead of feeding the ECG as a plain 1D wave, you turn each heartbeat into an "image," then use a 2D CNN — same kind of network that tells cats from dogs in photos.

Ways to make that image:

- Spectrogram — a heat-map of frequency changing over time
- Scalogram (wavelet-based) — similar idea, catches patterns at different time-scales, good fit for arrhythmias
- Gramian Angular Field — folds the 1D wave into a 2D pattern that still keeps the timing info
- Stack both ECG leads together like two "color channels" of one image

Why it's worth trying: 2D CNNs are extremely well-studied, you can even borrow a pretrained image model, and some irregular patterns are easier to spot visually than in a raw squiggly line. Downside: more preprocessing steps and slightly heavier compute — still totally fine on free Colab for this dataset size.

Treat it as one more entry in your comparison list, not a replacement for the rest.

On the project flow

Your skeleton is right — here it is cleaned into order:

1. Notebook setup — import numpy, pandas, matplotlib, wfdb (reads MIT-BIH files directly), scikit-learn, tensorflow/keras (or pytorch)
2. Load raw records, glance at a few signals, count how many beats fall in each class (bar chart) — look before touching anything
3. Preprocess — filter noise, find R-peaks, cut out individual heartbeats, normalize

4. Split into train/test by patient, not by random beats — so nobody leaks across both sides
5. Visualize both sets — class balance chart and a few sample waveforms for train and for test, as a sanity check
6. Build your algorithm lineup, the thing you'll actually compare:
   - Logistic regression / linear baseline
   - Random Forest or XGBoost, on handcrafted features (RR interval, shape)
   - 1D-CNN
   - LSTM / GRU
   - CNN-LSTM hybrid
   - 2D-CNN on spectrogram or GAF image
   - CNN-Transformer hybrid
   - Autoencoder anomaly detector — your original outlier idea
7. Train every model on the same training set, test every model on the same untouched test set
8. Confusion matrix for each one — but read past accuracy, check which classes get missed. A missed V or S beat (false negative) matters far more than a false alarm
9. Line all the confusion matrices up side by side, pick the strongest performer(s), maybe wire two together into the cascade from the diagram above

Each numbered step is basically one notebook section, in order.

Claude

1D-CNN

Definition: a network that slides small filters directly across the raw waveform, each filter learning to detect one specific shape — a sharp spike, a wide bump, a steep drop — wherever it appears in the signal.

Math, kept simple: each filter does a sliding dot-product, `output[i] = Σ (filter[k] × signal[i+k])`, then a nonlinearity (ReLU) keeps only the strong matches. Stack a few layers and early filters learn tiny shapes, later filters combine those into whole-beat patterns.

In this project: this works directly on Track A, the raw waveform — no handcrafted features needed. It learns what a PVC's wide QRS complex looks like on its own, which is exactly why CNNs are the backbone of nearly every strong MIT-BIH result in the literature.

Picture it: a raw ECG squiggle at the top, a small sliding window box scanning left to right underneath it, and beneath that a shorter row of "activation" bars lighting up wherever the window passes over a spike.

LSTM / GRU

Definition: a network built for sequences — it reads the signal step by step and carries a running "memory" forward, deciding at each point what to keep, what to forget, and what's currently relevant.

Math, kept simple: at each step it computes gates — `forget_gate`, `input_gate`, `output_gate` — each just a sigmoid between 0 and 1 controlling how much old memory survives versus how much new information gets written in. GRU is the same idea with fewer gates, so it's lighter and trains faster.

In this project: a single beat's *shape* is well handled by a CNN, but arrhythmias sometimes only reveal themselves across a *run* of beats — an irregular rhythm is a pattern in the sequence, not in any one beat alone. LSTM/GRU is what lets your model reason about that.

CNN-LSTM hybrid

Definition: run the CNN first to turn each raw beat into a compact set of learned shape-features, then feed that sequence of feature-summaries into an LSTM to catch the timing pattern across beats.

Math, kept simple: no new equation — it's the same two equations above, chained. CNN output becomes LSTM input.

In this project: this is the one nearly every 2025-2026 paper on MIT-BIH points to as the strongest single architecture, and it's why it's your Stage 2 classifier — CNN reads *what* each beat looks like, LSTM reads *how* beats relate to each other over time, combined into one model.

Picture it: a row of raw beats going into a CNN block ("shape reader"), its output feeding into an LSTM block ("sequence reader"), then a final arrow into the predicted class.

Next batch: FFT-based 2D-CNN, CNN-Transformer hybrid — say the word when ready.

User

go and finish it lastly giving image prompt for nano banana for all the Visualisation generation. Make it utterly descriptive for all the Algorithms

Claude

FFT-based 2D-CNN

Definition: turn each heartbeat into a 2D time-frequency image using a Short-Time Fourier Transform, then run a standard image-recognition CNN on that image instead of on the raw wave.

Math, kept simple: STFT slides a small window across the beat and takes an FFT of each window, stacking results into an image where x = time, y = frequency, brightness = energy at that time-frequency point.

In this project: this is Track C, and it's your Fourier idea done properly — different AAMI classes likely show different frequency signatures (a wide PVC skews toward lower frequencies than a sharp normal QRS), and the CNN can now see *when* those frequencies occur, not just that they exist.

CNN-Transformer hybrid

Definition: same CNN shape-reader as before, but instead of an LSTM carrying memory step-by-step, a Transformer lets every beat directly "attend to" every other beat at once, weighted by relevance.

Math, kept simple: `attention = softmax(QK^T/√d)V` — each beat's Query is compared against every other beat's Key, producing weights that decide how much of each beat's Value gets blended in.

In this project: this is why it tops most current MIT-BIH leaderboards — LSTMs can lose information over long stretches, attention connects distant beats directly, which helps on irregular, non-periodic rhythms.

Fusion model

Definition: run two branches in parallel — the 2D-CNN reading the spectrogram, and a small dense layer reading your handcrafted RR-interval/FFT features — then merge (concatenate) both outputs right before the final decision.

Math, kept simple: `combined = [CNN_output ; handcrafted_features]` → one final dense layer + softmax on the joined vector.

In this project: this directly tests whether your engineered FFT features add anything the deep image path hasn't already learned on its own — a clean, honest way to settle that question with numbers instead of guessing.

Ensemble

Definition: train several of your strongest individual models completely independently, then combine their final answers by vote or by averaging their probabilities.

Math, kept simple: `p_final = average(p_1, p_2, ..., p_N)`, or simplest of all, majority vote across predicted classes.

In this project: different models fail on different beats — CNN misses what attention catches, classical ML misses what deep learning catches — so combining your top 2-3 usually catches more than any one alone. This is your closing "best of everything" model.

Nano Banana prompts — one per algorithm

Shared style line to prepend to each (keeps the whole set visually consistent as a deck): *"Flat vector educational infographic style, dark navy background, clean sans-serif labels, muted teal/purple/coral/amber accent colors, no photorealism, minimalist medical-tech aesthetic, high clarity, 16:9."*

1. **Logistic regression** — "A scatter plot on a dark background: teal dots clustered lower-left labeled 'normal heartbeats', coral dots clustered upper-right labeled 'abnormal heartbeats', a single dashed white diagonal line cutting between the two clusters labeled 'decision boundary', small S-curve sigmoid graph in the corner showing probability from 0 to 1."

2. **Random Forest** — "A single glowing icon of a heartbeat waveform at the top labeled 'beat features', three small stylized tree-branch icons below it side by side labeled 'Tree 1, Tree 2, Tree 3' each with tiny yes/no branch splits, arrows converging downward into a ballot-box icon labeled 'majority vote', final labeled output box below it."
3. **Autoencoder** — "A horizontal sequence: a wide vertical bar of ECG waveform labeled 'input beat' on the left, narrowing through a funnel shape labeled 'encoder' into a single thin glowing bottleneck bar labeled 'compressed representation', then widening back out through a mirrored funnel labeled 'decoder' into a reconstructed wide bar on the right, with a red warning icon and arrow pointing to a box labeled 'large gap = anomaly flagged'."
4. **1D-CNN** — "A raw ECG waveform line stretching left to right across the top, a small rectangular sliding-window box scanning along the wave with a subtle motion trail, beneath it a shorter row of vertical activation bars lighting up in amber wherever the window passes over a sharp spike, labeled 'learned shape filters'."
5. **LSTM/GRU** — "A horizontal chain of small connected memory-cell icons (rounded squares) in a row, each cell showing three small gate symbols (forget, input, output) as tiny switch icons, a thin glowing thread of 'memory' flowing left to right through all the cells, ECG beat icons feeding into each cell from below, labeled 'sequence memory'."
6. **CNN-LSTM hybrid** — "Two connected blocks side by side: left block is a small CNN icon (stacked filter layers) labeled 'shape reader' processing a row of raw ECG beats, its output feeding into a right block of connected memory-cell icons labeled 'sequence reader', a final arrow pointing to a labeled output box 'predicted arrhythmia class'."
7. **FFT-based 2D-CNN** — "A raw ECG wave on the left transforming through a small window-sliding animation into a colorful heatmap image on the right (a spectrogram: horizontal axis time, vertical axis frequency, warm amber/red blotches showing high energy regions), the heatmap then flowing into a small stack of CNN filter-layer icons labeled 'image classifier'."
8. **CNN-Transformer hybrid** — "A horizontal row of heartbeat icon tokens, one highlighted token in amber at the center with thin glowing lines fanning out to every other token, line thickness varying to show attention weight strength, faint lines to distant tokens and thick bright lines to closely related ones, labeled 'self-attention across beats'."
9. **Fusion model** — "Two parallel vertical towers converging into one funnel shape at the bottom: left tower is a colorful spectrogram image flowing downward through small CNN block icons, right tower is a short table of numbers (RR interval, FFT power) flowing through a small dense-layer icon, both arrows merging into one glowing 'combined decision' box at the bottom center."
10. **Ensemble** — "Three small model icons in a row (a tree icon, a neural network icon, a wave icon), each producing its own small probability bar chart above it for the same heartbeat, thin arrows from all three bar charts merging into one larger blended bar chart at the bottom with a checkmark highlighting the winning class, labeled 'ensemble vote'."